

Stage 5. Instruction Set Architecture

❖ Arithmetic Logic Unit (ALU)

- Arithmetic Operations
- Logic Operations
- Multiplication
- Division

❖ A Best Practice for Our Basic Computer System

Arithmetic Logic Unit (ALU)

The ALU unit in a microprocessor is where all arithmetic and logical operations are performed. The ALU designed in our basic computer system is designed to perform operations on 16-bit data, except for multiplication and input/output operations. Since the registers used were 16-bit, two 8-bit numbers were used in the multiplication operation.

The designed ALU consists of four parts;

1. The section where other arithmetic operations are performed, except multiplication and division operations
2. The section where logical operations are performed
3. The section where multiplication takes place
4. The section where division is performed

Arithmetic Operations

In this section, 16 full adder were used. The design steps of the full adder are shown below.

Full Adder

The full adder takes 2 1-bit data and the carry bit (c_i) as input and produces the sum and carry output (c_o).

Inputs			Outputs	
AC	DR	c_i	Total	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

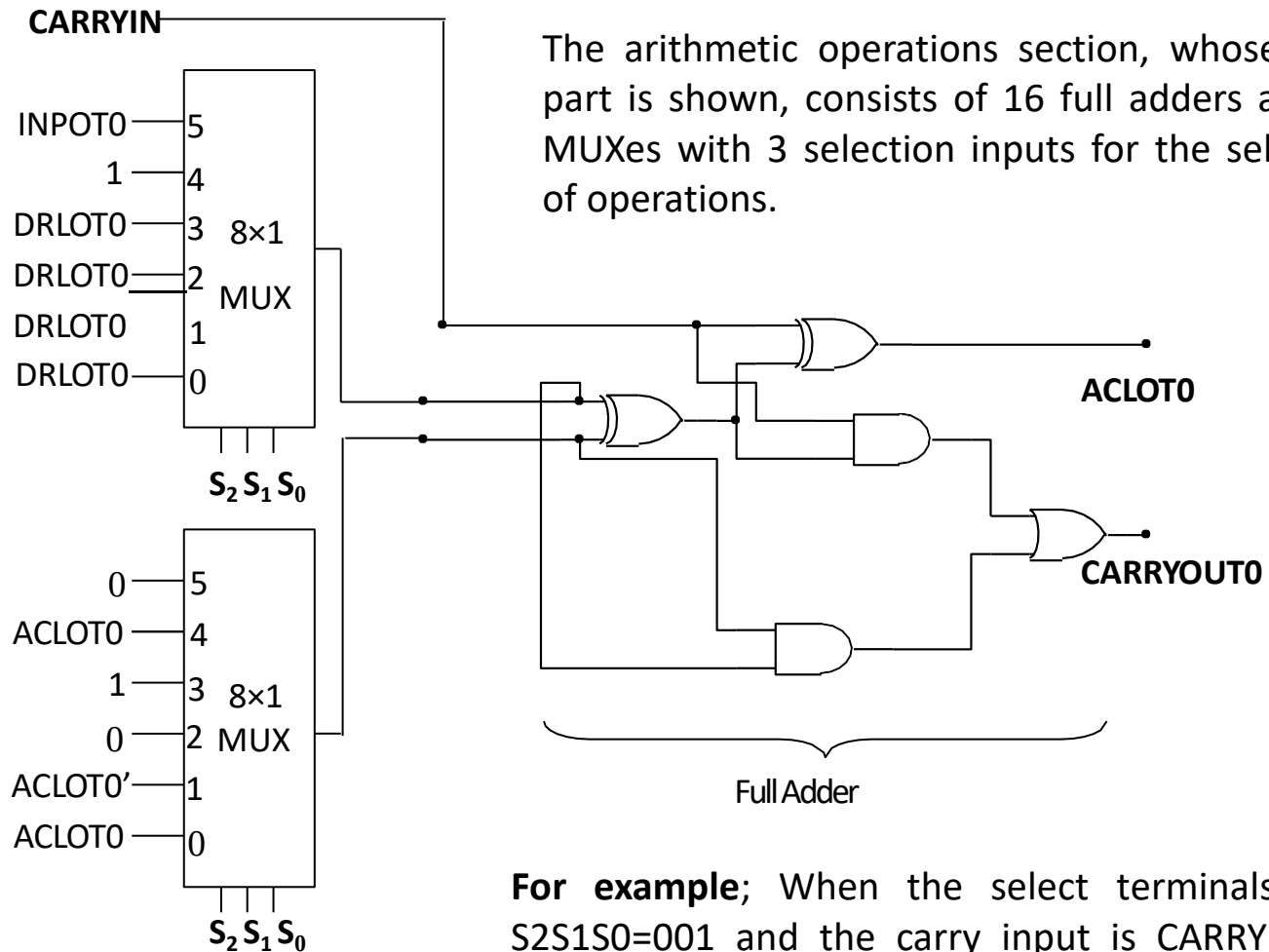
$$\begin{aligned}\text{Total} &= AC'.DR'.c_i + AC'.DR.c_i' + AC.DR'.c_i' + AC.DR.c_i \\ &= c_i.(AC'.DR' + AC.DR) + c_i'.(AC'.DR + AC.DR') \\ &= c_i.(AC \oplus DR)' + c_i'.(AC \oplus DR) \\ &= AC \oplus DR \oplus c_i\end{aligned}$$

$$\begin{aligned}c_o &= AC'.DR.c_i + AC.DR'.c_i + AC.DR.c_i' + AC.DR.c_i \\ &= c_i.(AC \oplus DR) + AC.DR\end{aligned}$$

Arithmetic Operations Performed on the ALU

Selection			Inputs			Output	Description
S_2	S_1	S_0	Carry(in)	Y_0	Y_1	Q	
0	0	0	0	DR	AC	$AC \leftarrow DR + AC$	Addition
0	0	0	1	DR	AC	$AC \leftarrow DR + AC + 1$	Addition with carry
0	0	1	0	DR	AC'	$AC \leftarrow DR + AC'$	Subtraction with debt
0	0	1	1	DR	AC'	$AC \leftarrow DR + AC' + 1$	Subtraction
0	1	0	0	DR	0..0	$AC \leftarrow DR$	Transfer of DR
0	1	0	1	DR	0..0	$AC \leftarrow DR + 1$	Transfer 1 more than DR
0	1	1	0	DR	1..1	$AC \leftarrow DR - 1$	Transferring 1 minus DR
0	1	1	1	DR	1..1	$AC \leftarrow DR$	Transfer of DR
1	0	0	0	1..1	AC	$AC \leftarrow AC - 1$	Decrease AC by 1
1	0	0	1	1..1	AC	$AC \leftarrow AC$	Transfer of AC
1	0	1	0	INPR	0..0	$AC \leftarrow INPR$	Transfer of Input Register
1	0	1	1	INPR	0..0	$AC \leftarrow INPR + 1$	Transferring 1 more of the Input Register

Implementation of the 1-Bit Part of the Arithmetic Operations Section



For example; When the select terminals are $S_2S_1S_0=001$ and the carry input is $CARRYIN=1$, the contents of the accumulator are subtracted from the contents of the data register.

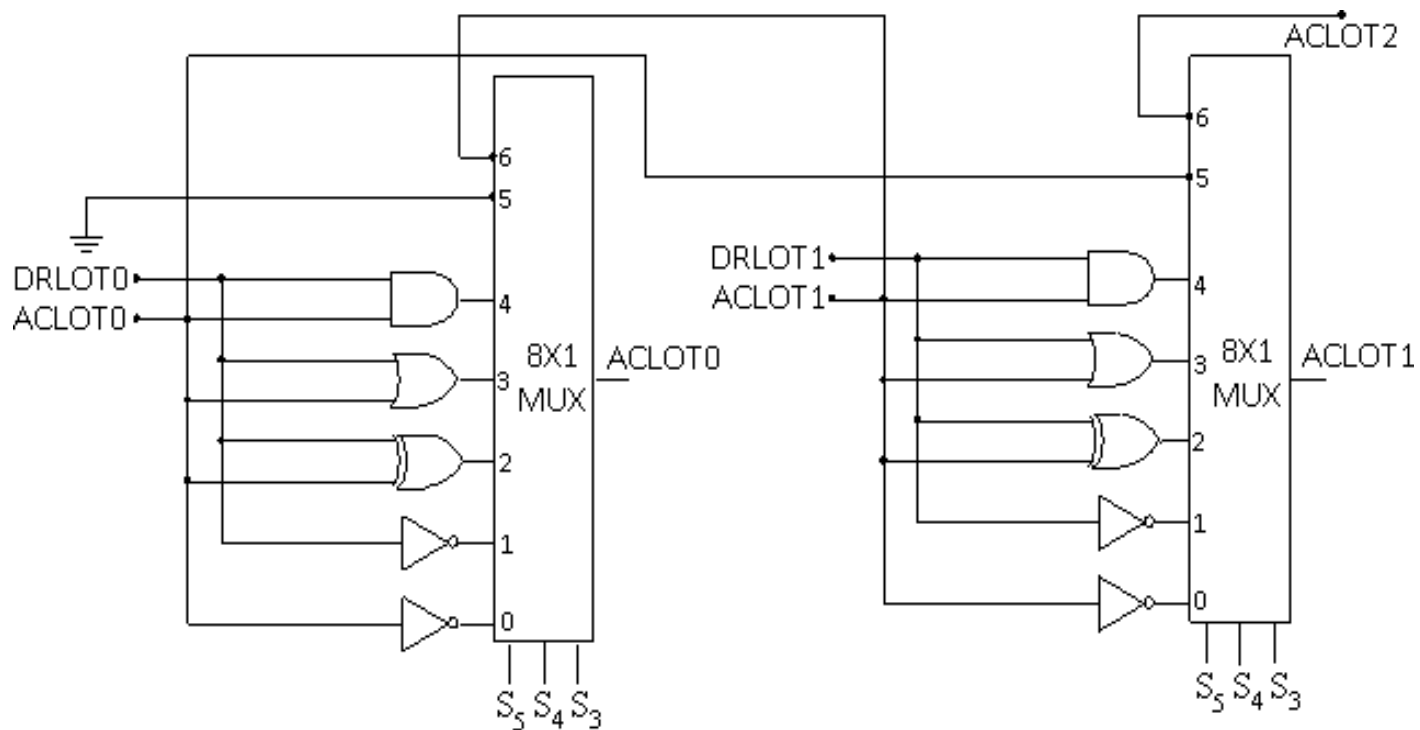
Logical Operations

This part can perform 7 different operations. The operations performed are given in the table below;

S_5	S_4	S_3	Q	Description
0	0	0	$AC \leftarrow AC'$	Complement of AC
0	0	1	$AC \leftarrow DR'$	Complement of DR
0	1	0	$AC \leftarrow AC \oplus DR$	Logic "XOR" Operation
0	1	1	$AC \leftarrow AC \vee DR$	Logic "OR" Operation
1	0	0	$AC \leftarrow AC \wedge DR$	Logic "AND" Operation
1	0	1	$AC \leftarrow shl(AC)$	Shifting AC left
1	1	0	$AC \leftarrow shr(AC)$	Shifting AC right

Implementation of the 2-Bit Part of the Logical Operations Section

16 8x1 MUX units were used to perform the operations performed in this section of the ALU.

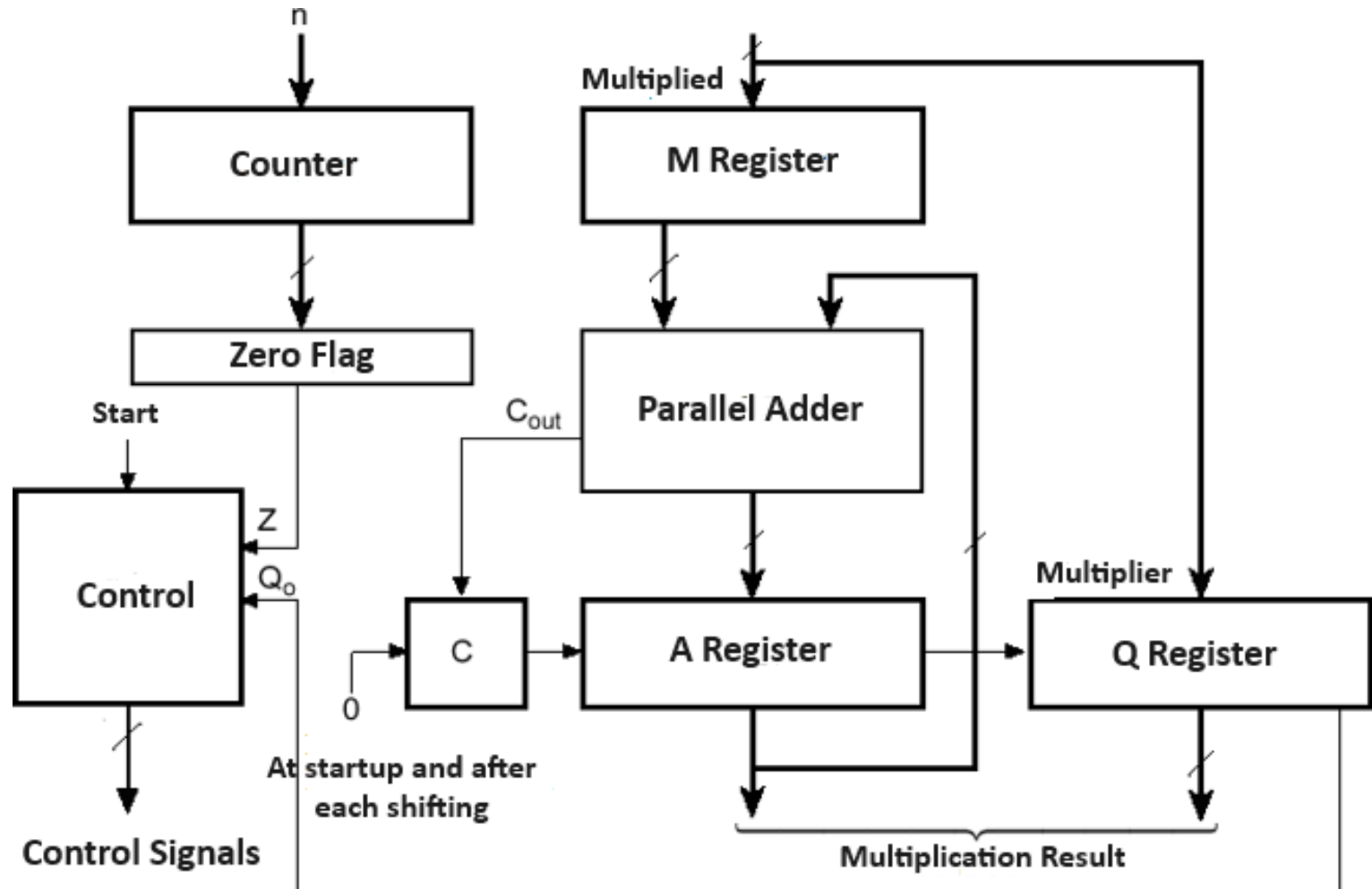


For example; When the selection terminals are $S_5S_4S_3 = 011$, the data in the data recorder and the data in the accumulator are transferred to the accumulator by subjecting them to the logical “OR” operation.

Multiplication

- Multiplication of two binary numbers, if done with pen and paper, is accomplished through a series of sequential shifting and addition operations. The process is done by looking at all the bits of the multiplier in order, starting from the least significant bit. If the bit of the multiplier is 1, the multiplier is taken down exactly as it is. Otherwise, zeros are written below. It is then shifted one bit to the left. By adding the partial products, the product is obtained.
- When hardware multiplication is to be performed, instead of writing all partial products to registers, the sum of partial products is written to a register. The partial sum is then shifted to the right. If the multiplier bit is 0, there is no need to add with zero, just shift to the right.

Principle Diagram of Multiplication



Example: Implementing the Algorithm for Operation 1001×0011

- The value of the multiplicand is placed in register M and the value of the multiplier is placed in register Q.
- Register A and carry bit (c) are reset, and the counter is assigned the value 4.
- The sum of partial products is obtained by looking at the Q0 bit of the multiplier.
- The high-significance part of the result of the operation is in register A, and the low-significance part is in register Q.

M = 1001

Initial Values	C(0)	A (0000)	Q (0011) Q0	Operation
1.Step	0	1001	0011	Addition
	0	0100	1001	Shifting
2. Step	0	1101	1001	Addition
	0	0110	1100	Shifting
3. Step	0	0011	0110	Shifting
4. Step	0	0001	1011	Shifting

Multiplication = A Q = 0001 1011 = $1B_h = 27$

Division

- Dividing two binary numbers requires comparison, shift and subtraction operations. Binary division is simpler than decimal division; The digits of the numbers being processed have only 0 or 1 in their digits. It is also easy to find out how many times the divisor appears in the dividend or remainder.
- If we want to perform the division operation hardware on a digital computer, the operations will change slightly. A division or partial remainder is shifted to the left. If there is a divisor in the dividend, the necessary subtraction is done by adding the complement of the divisor to the dividend according to 2.

Algorithm of the Division

The divisor is put in the M register, and the dividend is put in the Q register.

The ordinal counter is initially assigned the bit number of the divisor.

The divisor is shifted to the left by 1, and the most significant part of the divisor is transferred to its register (initially $A = 0$) and the 2's complement of the divisor is added to A. If the result of this operation is positive, 1 is written in the quotient (1 is placed in the least significant part of Q) and the row counter is decreased by 1. If the remainder is negative, it means that there is no divisor in the dividend (0 is placed in the least significant part of Q). The dividend 1 is shifted to the left and the partial remainder is restored by adding the divisor. Again, it is investigated whether the divisor has a divisor or not. These operations continue until the counter is reset.

Example: Implementation of the Algorithm for Transaction 010110/000011

- The value of the divisor is placed in the M register, and the value of the dividend is placed in the Q register.
- Register A keeps the remainder resulting from the division operation and is initially assigned a value of zero, and a value of 6 is loaded into the counter.
- Register Q is shifted to the left by 1 towards register A. If there is a divisor in A, 1 is loaded into the 0th bit of Q, otherwise 0 is loaded. These operations are performed 6 times.

Initial Values	A (000000)	Q (010110) Q0	Operations	M= 000011
1.Step	000000	10110 0	Shifting	
2. Step	000001	01100 0	Shifting	
3. Step	000010	11000 0	Shifting	
4. Step	000101	100000	Shifting	
	000010	10000 1	Substraction	
5. Step	000101	000010	Shifting	
	000010	00001 1	Substraction	
6. Step	000100	000110	Shifting	
	000001	00011 1	Substraction	

Division = Q = 000111₂ = 7

Remainder = A = 000001₂ = 1

Example

Finding assembly and machine language equivalents of the following program, written in a high-level language, according to our basic computer system.

```
a = 5;  
for i = 1 to 10  
{  
    a = a+i;  
}
```

Variables are put into memory starting at address 0030h;

0030h-0031h: a variable

0032h-0033h: i variable

0034h-0035h: max value of i variable

Finding the Assembly Language Equivalent of the Program

0000h: LDA #0005h / The accumulator is assigned the decimal value 5, which will be put into variable a.
0003h: STA 0030h / For variable a, memory addresses 0030h to 0031h are allocated.
0006h: LDA #0001h / The accumulator is assigned a decimal value of 1, which will be placed in variable i.
0009h: STA 0032h / For variable i, memory addresses 0032h and 0033h are allocated.
000Ch: LDA #000Ah / Decimal 10, which is the maximum value of variable i, is assigned to the accumulator.
000Fh: STA 0034h / i'nin maksimum değeri için, belleğin 0034h ile 0035h adresleri tahsis edilmiştir.
0012h: LDA 0030h / For the maximum value of i, addresses 0034h to 0035h of memory are allocated.
0015h: ADD 0032h / Variable a and variable i are added.
0018h: STA 0030h / The total result is saved in the memory area where variable a is kept.
001Bh: LDA 0032h / Variable i is retrieved from memory.
001Eh: SUB 0034h / i is subtracted from the maximum value of i (decimal 10).
0021h: BZR ~09h / If variable i reaches the decimal value 10, it branches to address 002Ch.
0023h: LDA 0032h / Variable i is retrieved from memory.
0026h: INCR / The accumulator to which the value of variable i is transferred is incremented by 1.
0027h: STA 0032h / Variable i is being saved into memory.
002Ah: BRA ~E6h / Branching to address 0012h.
002Ch: LDA 0030h / The total result is transferred to the accumulator (to see the result)
002Fh: HLT / The program is terminating.

Finding the Machine Language Equivalent of the Program

MEMORY ADDRESSES	MACHINE LANGUAGE	ASSEMBLY LANGUAGE
0000	1A	LDA #0005h
0001	00	
0002	05	
0003	A0	STA 0030h
0004	00	
0005	30	
0006	1A	LDA #0001h
0007	00	
0008	01	
0009	A0	STA 0032h
000A	00	
000B	32	
000C	1A	LDA #000Ah
000D	00	
000E	0A	
000F	A0	STA 0034h
0010	00	
0011	34	
0012	2A	LDA 0030h
0013	00	
0014	30	

0015	20	ADD 0032h
0016	00	
0017	32	
0018	A0	STA 0030h
0019	00	
001A	30	
001B	2A	LDA 0032h
001C	00	
001D	32	
001E	2E	SUB 0034h
001F	00	
0020	34	
0021	53	BZR ~09h
0022	09	
0023	2A	LDA 0032h
0024	00	
0025	32	
0026	03	INCR
0027	A0	STA 0032h
0028	00	
0029	32	
002A	50	BRA ~E6h
002B	E6	
002C	2A	LDA 0030h
002D	00	
002E	30	
002F	0E	HLT

Example-2

```
k=5;  
m=10;  
s=0;  
if (k>=m)  
    s=k+m;  
else  
    s=k-m;
```

Variables are put into memory starting at address DE00h;

DE00h-DE01h: k variable

DE02h-DE03h: m variable

DE04h-DE05h: s variable

Example-2

```
0000h:LDA  #0005H;
0003h:STA  DE00H;
0006h:LDA  #000AH;
0009h:STA  DE02H;
000Ch:LDA  #0000H;
000Fh:STA  DE04H;
0012h:LDA  DE02H;
0015h:SUB  DE00H;
0018h:BCS  ~0AH;
001Ah:LDA  DE02H;
001Dh:SUB  DE00H;
0020h:STA  DE04H;
0023h:HLT
0024h:LDA  DE00H;
0027h:ADD  DE02H
002Ah:STA  DE04H
002Dh: HLT
```

```
k=5;
m=10;
s=0;
if (k>=m)
    s=k+m;
else
    s=k-m;
```

```
DE00h-DE01h:  k   variable
DE02h-DE03h:  m   variable
DE04h-DE05h: s variable
```

Example-2

BELLEK ADRESLERİ	MAKİNE DİLİ	ASSEMBLY DİLİ
0000	1A	LDA #0005h
0001	00	
0002	05	
0003	A0	STA DE00h
0004	DE	
0005	00	
0006	1A	LDA #000Ah
0007	00	
0008	0A	
0009	A0	STA DE02h
000A	DE	
000B	02	
000C	1A	LDA #0000h
000D	00	
000E	00	
000F	A0	STA DE04h
0010	DE	
0011	04	
0012	2A	LDA DE02h
0013	DE	
0014	02	

0015	2E	SUB DE00h
0016	DE	
0017	00	
0018	52	BCS ~0AH
0019	0A	
001A	2A	LDA DE02h
001B	DE	
001C	02	
001D	2E	SUB DE00h
001E	DE	
001F	00	
0020	A0	STA DE04H
0021	DE	
0022	04	
0023	0E	HLT
0024	2A	LDA DE00H
0025	DE	
0026	00	
0027	20	ADD DE02H
0028	DE	
0029	02	
002A	A0	STA DE04H
002B	DE	
002C	04	
002D	0E	HLT